MaratonaCIn

Centro de Informática UFPE

Universidade Federal de Pernambuco

SELETIVA AULA 4

Caminhos Mínimos

Árvore Geradora mínima

Disjoint Set Union

Caio Santos <cns>

Samuell Costa <shcpc>

rev. 1.1

CC BY NC SA

MaratonaCIn

Centro de Informática UFPE

Universidade Federal de Pernambuco

Caminhos Mínimos

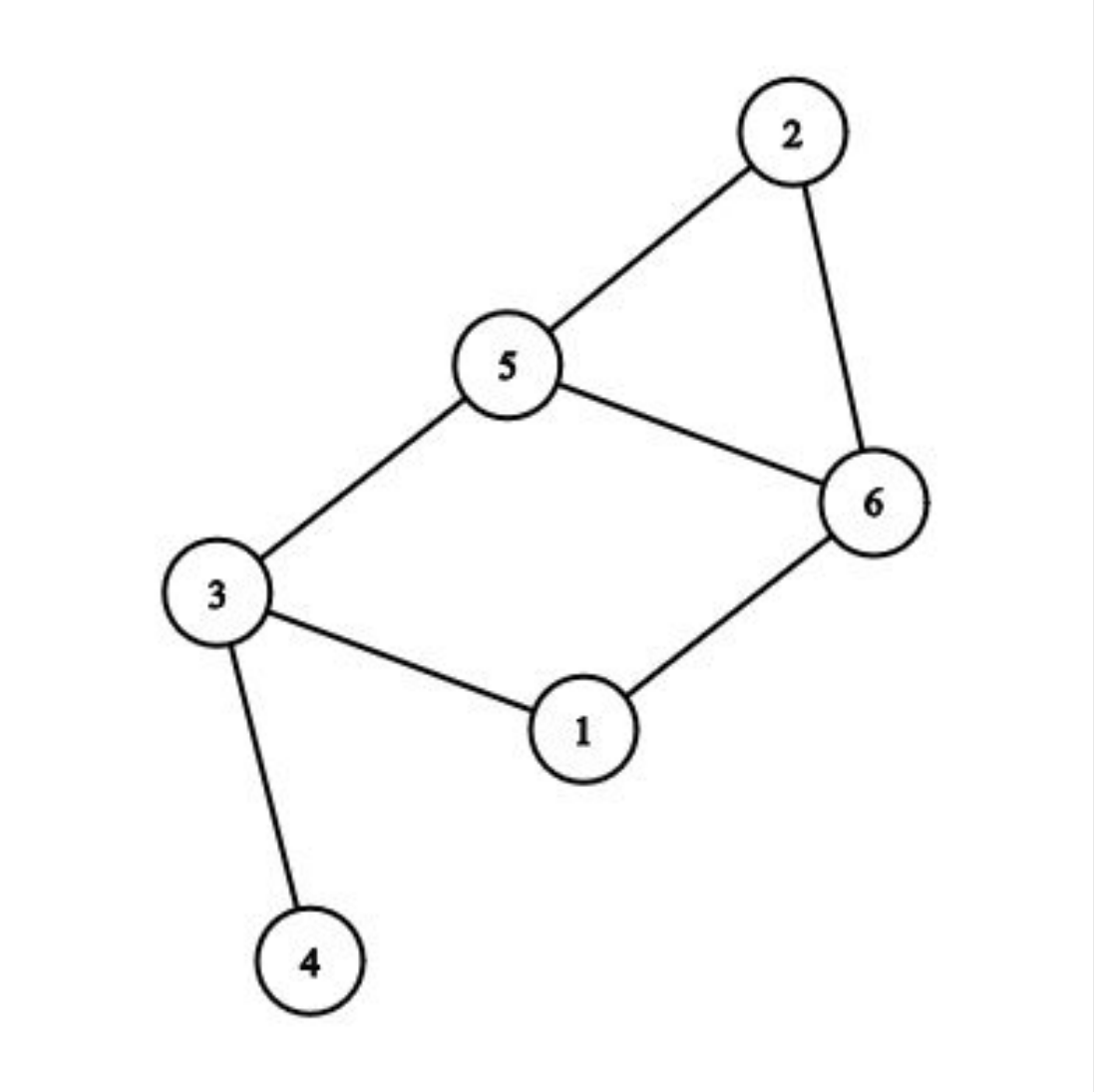
Problema

Encontre o menor caminho de um nó inicial para todos os outros.

Para esse grafo, é simples: **BFS!**

Por quê?

A chave para um algoritmo ótimo em um grafo: **não recalcular nós**

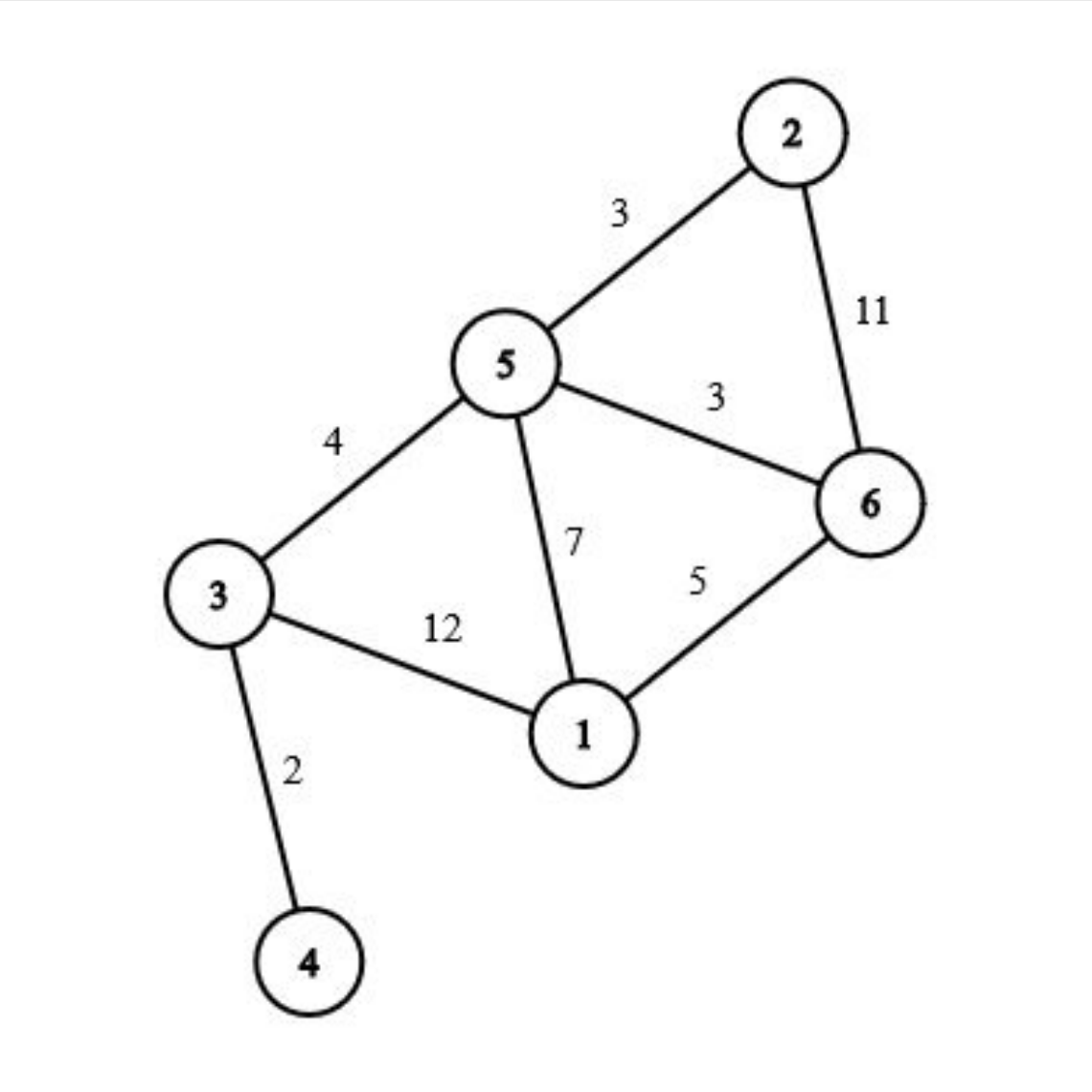


Problema

Menos “passos” não implica em menos custo.

É possível chegar em um nó pela primeira vez por um caminho pior.

Recalcular as distâncias quando necessário torna o algoritmo **exponencial**.



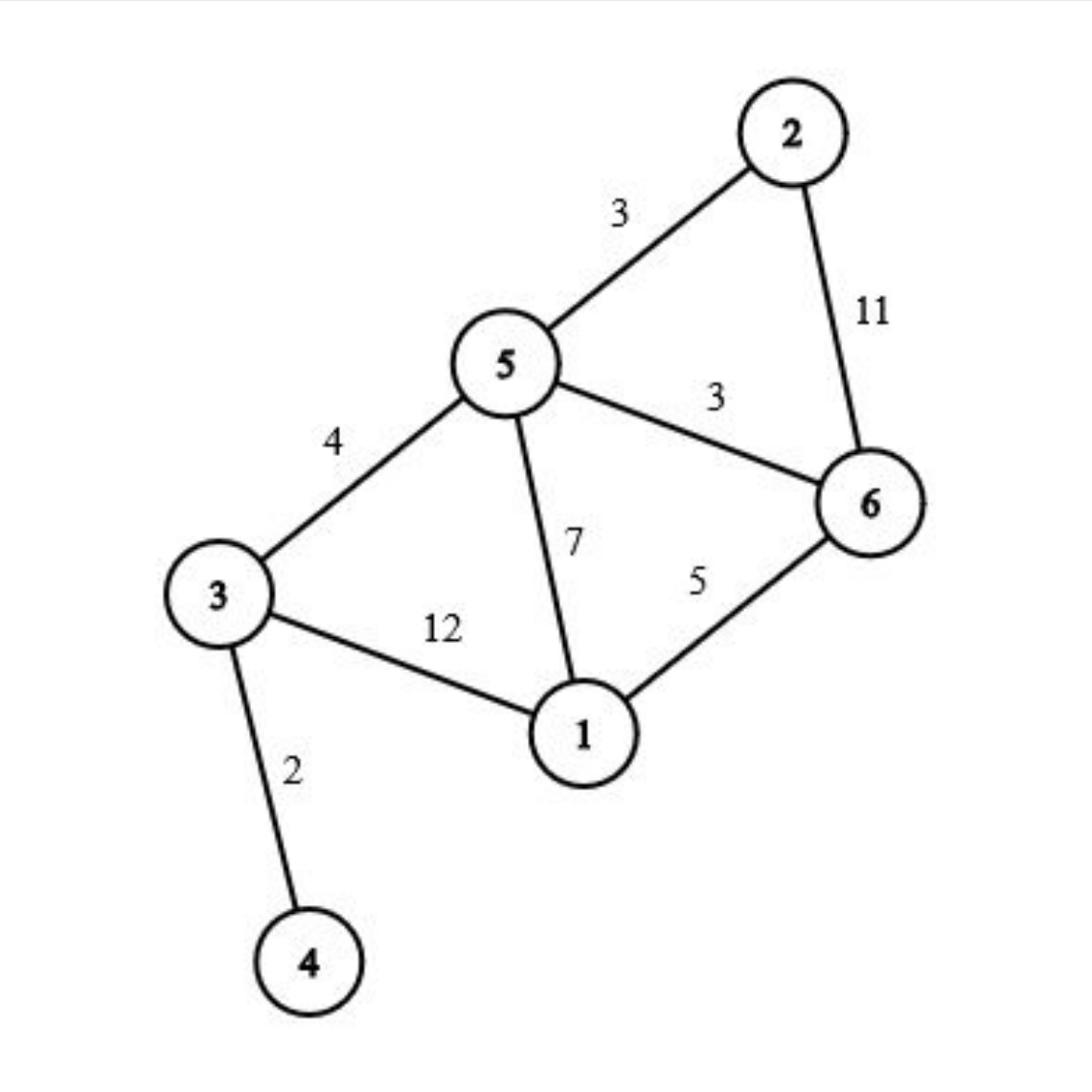
Problema

Estando no nó 1, é possível prever que escolha levará ao menor caminho até o nó 4?

E até o nó 2?

Nó 5? 6? 3?

"O subconjunto de uma solução ótima é também uma solução ótima para o subproblema que representa"



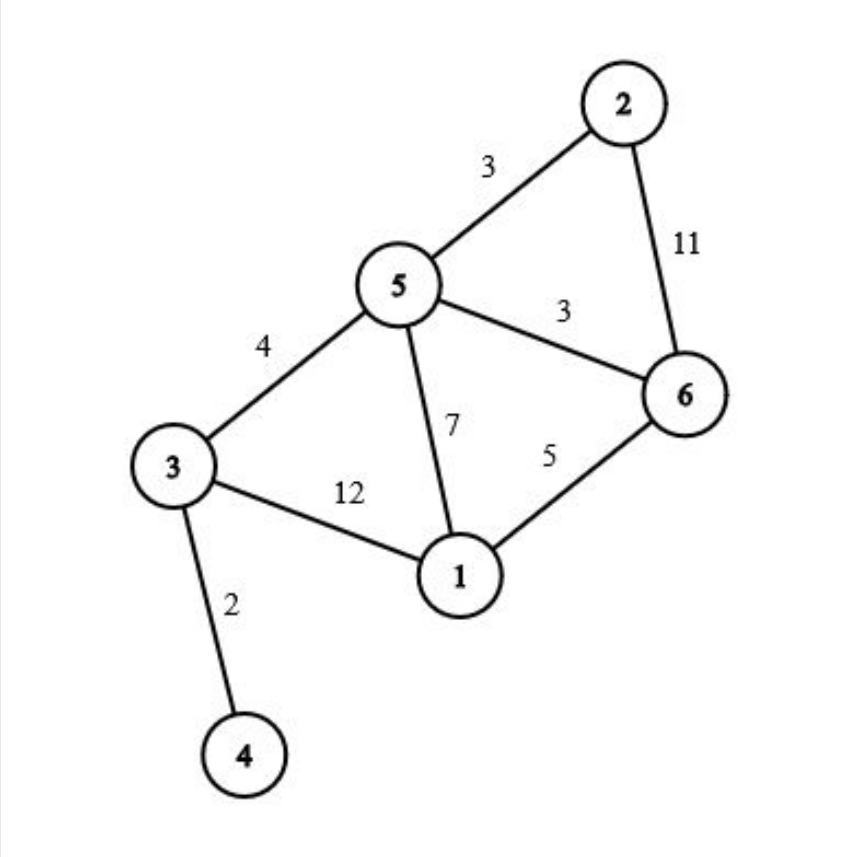
Problema

"O subconjunto de uma solução ótima é também uma solução ótima para o subproblema que representa"

Se algum menor caminho para outro nó passa pelo 6, necessariamente ele chegará no 6 pelo menor caminho de 1 até 6.

Dada as arestas disponíveis ao começar no nó 1, sabe-se que o menor caminho até 6 é o de custo 5. O contrário implica que alguma das outras arestas levaria para um caminho menor que 5, mas elas por si só já são maiores. **Essa é a menor distância final para ele.**

Agora, com as novas arestas que conheço posso chegar no nó 5 com custo 8 (3+5) ou 7.



Dijkstra

Colocar todos os possíveis menores caminhos conhecidos e seu custo total numa heap de mínimo.

Gulosamente escolher o melhor disponível e adicionar novas possibilidades quando define a distância de um vértice.

Complexidade: **O((V+E)logE)**

(V - vértices; E - arestas)

MaratonaCIn

Centro de Informática UFPE

Universidade Federal de Pernambuco

```
const int n = 1e5 + 10;
vector<ll> dist(n+1, LLONG_MAX);
vector<vector<tuple<ll, ll>>> adj(n+1);
priority_queue<tuple<ll, ll>,
    vector<tuple<ll, ll>>,
    greater<tuple<ll, ll>>> pq;

pq.push({0, 1});
dist[1] = 0;
while(!pq.empty()){
    auto [d, v] = pq.top();
    pq.pop();

    if(d <= dist[v]){
        for(auto [u, w] : adj[v]){
            if(dist[u] > dist[v] + w){
                dist[u] = dist[v] + w;
                pq.push({dist[u], u});
            }
        }
    }
}
```

Dijkstra

- Funciona para qualquer função de custo f onde:
  - se a <= b, f(a,w) <= f(b, w)
  - f(a, w) >= a
- Não funciona com arestas negativas
- Pode ser iniciado de várias fontes (multisource)

E se quiser a distância entre todos os pares?

O(V*(V+E)logE (geralmente O(V^3logV))

MaratonaCIn

Centro de Informática UFPE

Universidade Federal de Pernambuco

```
const int n = 1e5 + 10;
vector<ll> dist(n+1, LLONG_MAX);
vector<vector<tuple<ll, ll>>> adj(n+1);
priority_queue<tuple<ll, ll>,
    vector<tuple<ll, ll>>,
    greater<tuple<ll, ll>>> pq;

pq.push({0, 1});
dist[1] = 0;
while(!pq.empty()){
    auto [d, v] = pq.top();
    pq.pop();

    if(d <= dist[v]){
        for(auto [u, w] : adj[v]){
            if(dist[u] > dist[v] + w){
                dist[u] = dist[v] + w;
                pq.push({dist[u], u});
            }
        }
    }
}
```

Dijkstra

- O algoritmo não é caracterizado pela heap
- Originalmente iterava por todos os vértices disponíveis para encontrar o menor.
- Pode ser melhor dependendo dos limites do problema (grafos densos)

Complexidade: **$O(V^2)$**

Para todos os pares: **$O(V^3)$**

MaratonaCIn

Centro de Informática UFPE

Universidade Federal de Pernambuco

```cpp
const int n = 1e5 + 10;
vector<ll> dist(n+1, LLONG_MAX);
vector<bool> vis(n+1, false);
vector<vector<tuple<ll, ll>>> adj(n+1);
dist[1] = 0;
for(int i = 0; i < n; i++){
    v = -1;
    mn = LLONG_MAX;
    for(int j = 1; j <= n; j++){
        if((!vis[j]) && (dist[j] < mn)){
            mn = dist[j];
            v = j;
        }
    }
    if(v==-1) break;
    vis[v] = true;
    for(auto [u, w] : adj[v]){
        if(dist[u] > dist[v] + w){
            dist[u] = dist[v] + w;
        }
    }
}
```

Problema

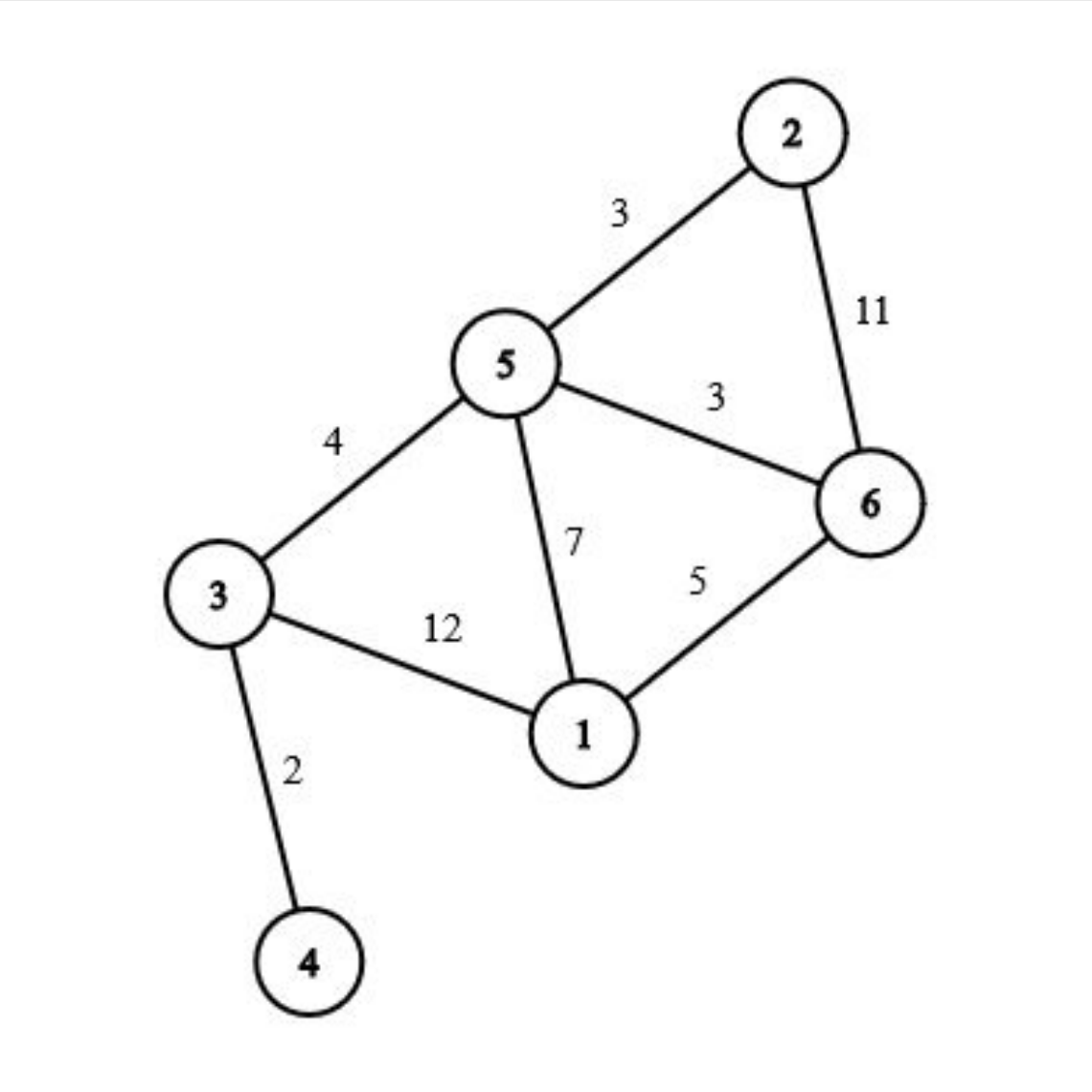
Como computar o menor caminho entre todos os pares de forma mais eficiente?

O primeiro passo é diminuir a escala do problema: **qual o menor caminho entre todos os pares que só use até o vértice 1?**

para cada par (u, v):

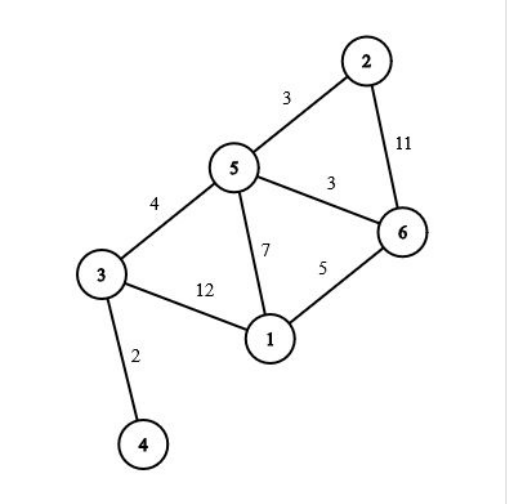
dist[u][v] = min(dist[u][v],

dist[u][1] + dist[1][v])



Indução

	A	B	C	D	E	F
1	0	INF	12	INF	7	5
2	INF	0	INF	INF	3	11
3	12	INF	0	2	4	INF
4	INF	INF	2	0	INF	INF
5	7	3	4	INF	0	3
6	5	11	INF	INF	3	0



Caso base: temos as distâncias para todos os pares (u, v) sem passar por nenhum outro vértice.

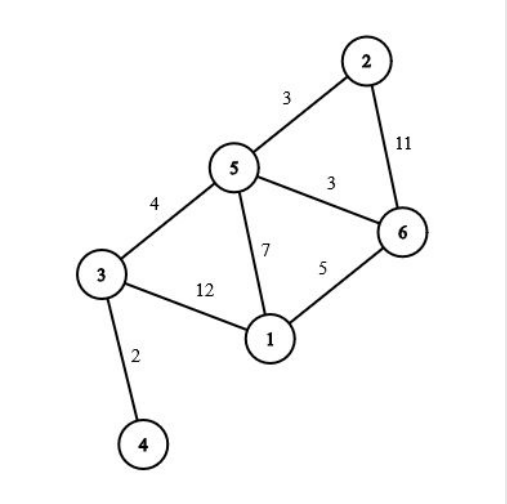
Passo indutivo: assuma que temos todas as distâncias (u, v) que passem por um subconjunto dos vértices até K.

Se existe um menor caminho (u, v) que passe por K+1, ele vai ser da forma:

u -> ... -> K+1 -> ... -> v

Indução

	A	B	C	D	E	F
1	0	INF	12	INF	7	5
2	INF	0	INF	INF	3	11
3	12	INF	0	2	4	INF
4	INF	INF	2	0	INF	INF
5	7	3	4	INF	0	3
6	5	11	INF	INF	3	0



Se existe um menor caminho (u, v) que passe por K+1, ele vai ser da forma:

u -> ... -> K+1 -> ... -> v

Se u -> ... -> K+1 não passa por nenhum acima de K, já temos dist[u][K+1] pelo passo anterior

Se K+1 -> ... -> v não passa por nenhum acima de K, igualmente

Logo: dist[u][v] = min(dist[u][v], dist[u][K+1] + dist[K+1][v])

Floyd-Warshall

Na K-ésima iteração gera os menores caminhos entre pares (u, v) que somente passem por um subconjunto de {1, 2, ..., K}.

(note que o subconjunto pode ser o conjunto vazio)

No final, terá considerado todos os vértices do grafo.

Complexidade: **$O(V^3)$**

(V - vértices)

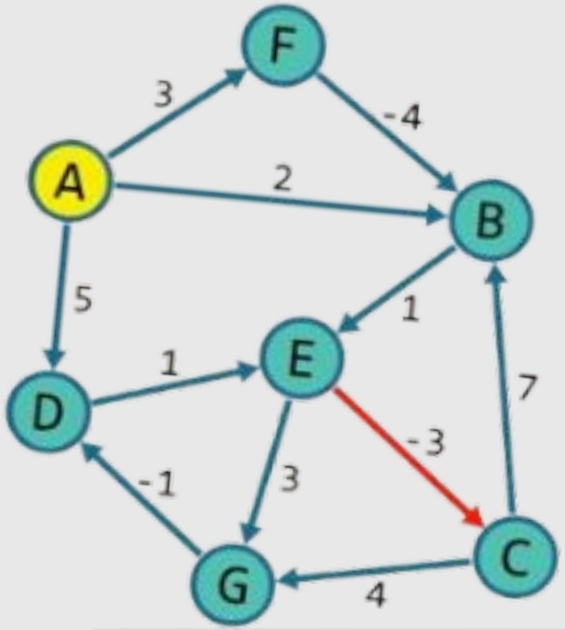
```
const int n = 300 + 10;
vector<vector<ll>> dist(n+1, vector<ll>(n+1));
//Cuidado com múltiplas arestas (pegue o mínimo)
//dist[i][j] deve ser 0 se (i==j)
for(int k = 1; k <= n; k++){
    for(int u = 1; u <= n; u++){
        for(int v = 1; v <= n; v++){
            if((dist[u][k] != LLONG_MAX) &&
            (dist[k][v] != LLONG_MAX)){
                dist[u][v] = min(dist[u][v],
                dist[u][k] + dist[k][v]);
            }
        }
    }
}
```

Pesos negativos

Para funcionar, o Dijkstra assume que todos os custos das arestas são >= 0.

Porém é possível ainda que um grafo possa ter arestas com **custos negativos**. Nesse caso podemos utilizar outro algoritmo, **Bellman ford.**

Existe ainda o problema dos ***ciclos negativos***, de forma que andar por esse ciclo faz o custo do caminho diminuir **infinitamente**. Bellman ford consegue identificar esses casos. Isso implica ainda que ***arestas negativas têm de ser direcionadas.***



Node	Cost	Previous
A	0	
B	-1	F
C	-3	E
D	0	G
E	0	B
F	3	A
G	1	C

Bellman-Ford

Consideramos a distância para todos os outros vértices como infinito e **a cada iteração testamos todas as arestas** tentando melhorar a menor distância de S para V.

No pior caso são necessárias **N-1 iterações.**

Para **identificar ciclos negativos**, podemos iterar uma N-ésima vez e se alguma distância diminuir então existe um ciclo negativo.

```
// Inicializar as distâncias
vector<ll> dist(n, INF);
dist[s] = 0;


// Bellman-Ford
for(int rep = 0; rep < n - 1; rep++)
  for(auto [u, v, w] : edges)
    dist[v] = min(dist[v], dist[u] + w);

```

```
//Ciclo negativo?
for(auto [u, v, w] : edges)
  if(dist[v] > dist[u] + w)
    cicloNegativo = true;
```

Caminhos Mínimos

MaratonaCIn Centro de Informática UFPE Universidade Federal de Pernambuco

	Dijkstra	Bellman-Ford	Floyd-Warshall
	O(N+M log N)	O(N * M)	O(N^3)
...	Caminho de um para todos. **Pesos positivos.**	Caminho de um para todos. **Pesos podem ser negativos.**	Caminho de **todos para todos.**

Aplicação

MaratonaCIn Centro de Informática UFPE Universidade Federal de Pernambuco

Flight Discount

TASK | STATISTICS

Time limit: 1.00 s **Memory limit:** 512 MB

Your task is to find a minimum-price flight route from Syrjälä to Metsälä. You have 10 discount coupons using which you can halve the price of any single flight during the route. However, you can only use the coupon once.

When you use the discount coupon for a flight whose price is x, its price becomes $\lfloor x/2 \rfloor$ (it is rounded down to an integer).

Output

Print one integer: the price of the cheapest route from Syrjälä to Metsälä.

MaratonaCIn

Centro de Informática UFPE

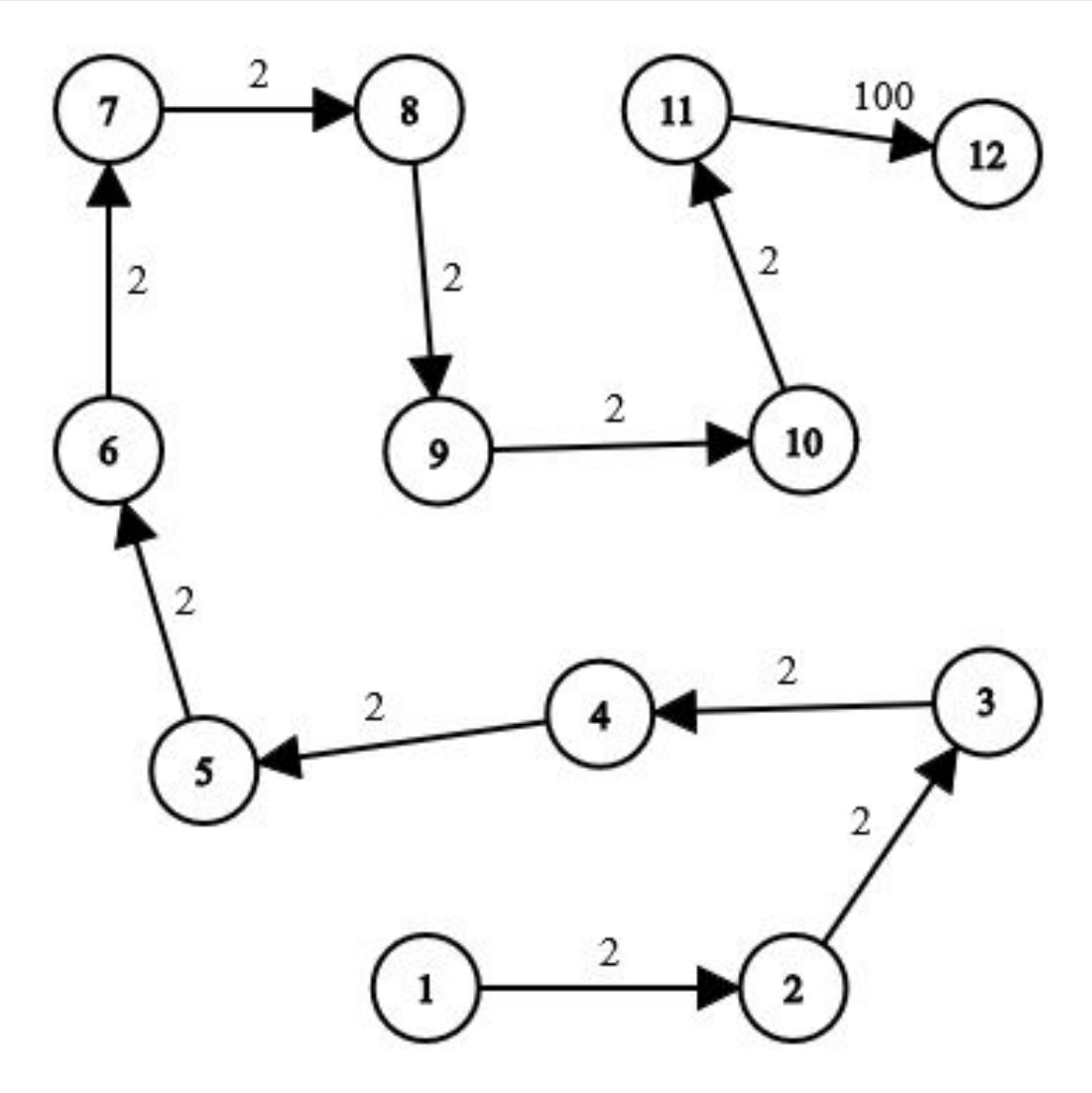
Universidade Federal de Pernambuco

Modelagem de Grafos

Problema

O Dijkstra padrão sempre verá a menor distância para [2, 3, ..., 11] como a usando todos os cupons.

Não podemos recalcular vértices, as possibilidades de não usar cupom imediatamente serão ignoradas na heap.

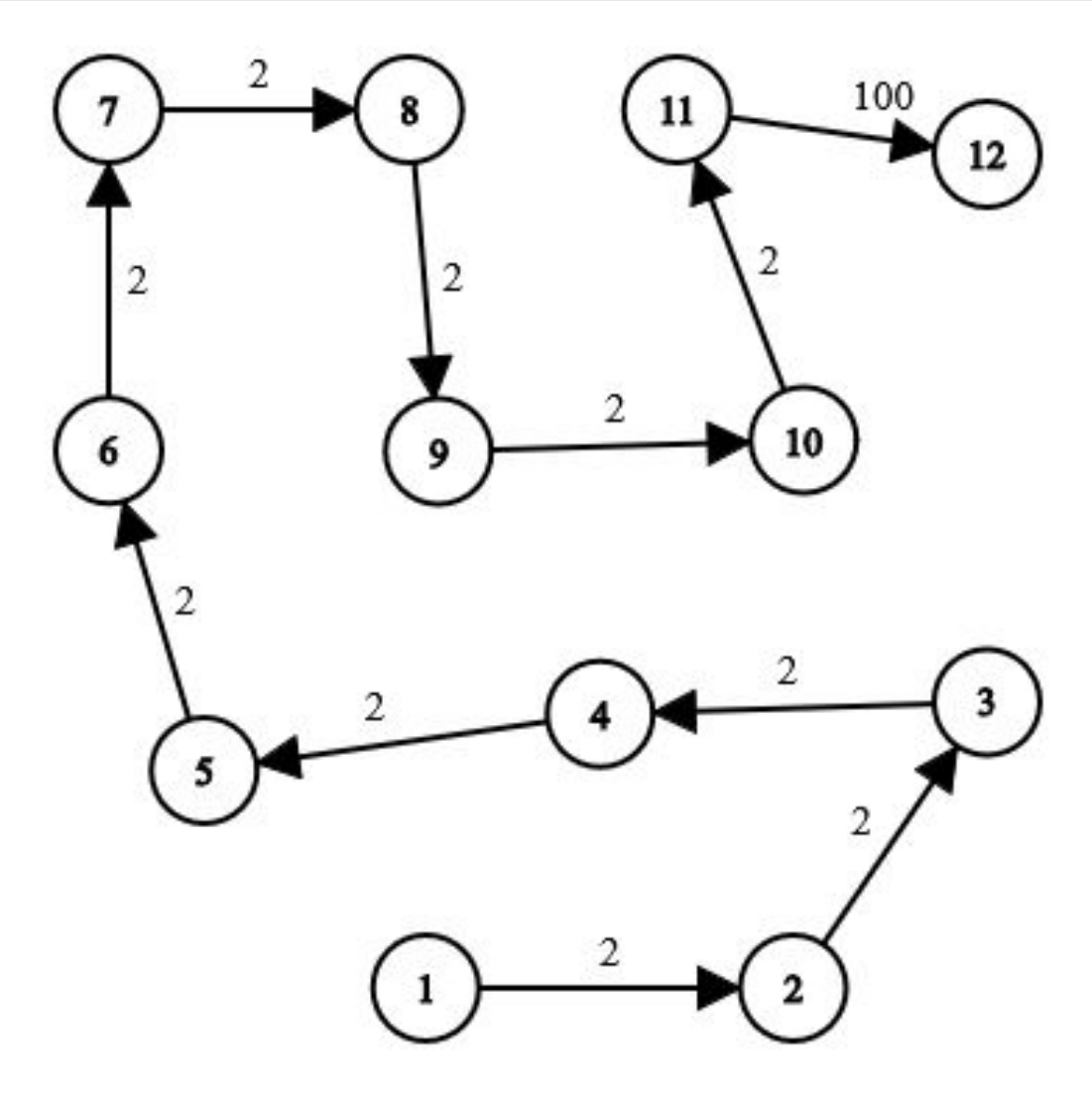


Problema

Um vértice não descreve o subproblema.

A menor distância para chegar em 2 com um cupom gasto é 1. Mas, na melhor resposta, devemos chegar nele sem gastar nada.

A quantidade de cupons usados influencia a resposta. **Como representar?**

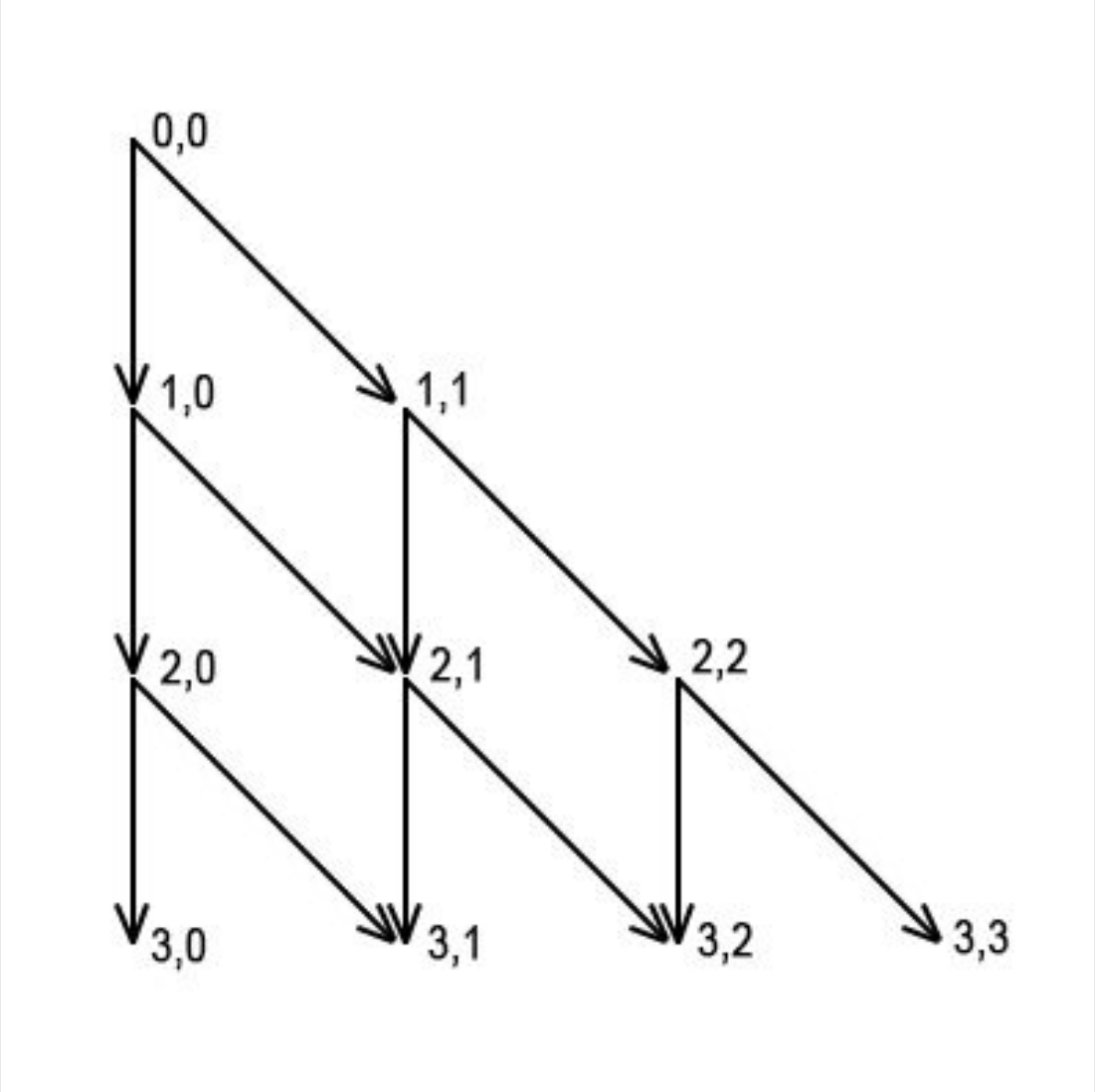


Problema

Estado: conjunto de atributos necessários e suficientes para **identificar um subproblema** e computar sua resposta pelas **transições**.

“Escolhas” diferentes não devem existir no mesmo estado.

Transição: nesse caso, as arestas para seus vizinhos.



Modelagem

Modelagem é transformar o problema em outro grafo que represente os estados do problema e permita a utilização dos algoritmos clássicos.

lembrem disso pra DP!!!!

Como isso afeta a complexidade?

V = número de estados

(10*N), nesse caso

```
while(!pq.empty()){
        auto [d, v, c] = pq.top();
        pq.pop();

        if(d <= dist[v][c]){
            for(auto [u, w] : adj[v]){
                if(dist[u][c] > dist[v][c] + w){
                    dist[u][c] = dist[v][c] + w;
                    pq.push({dist[u][c], u, c});
                }
                if(c < 10){
                    if(dist[u][c+1] > dist[v][c] + (w/2)){
                        dist[u][c+1] = dist[v][c] + (w/2);
                        pq.push({dist[u][c+1], u, c+1});
                    }
                }
            }
        }
    }
```

MaratonaCIn

Centro de Informática UFPE

Universidade Federal de Pernambuco

Disjoint Set Union

--

DSU

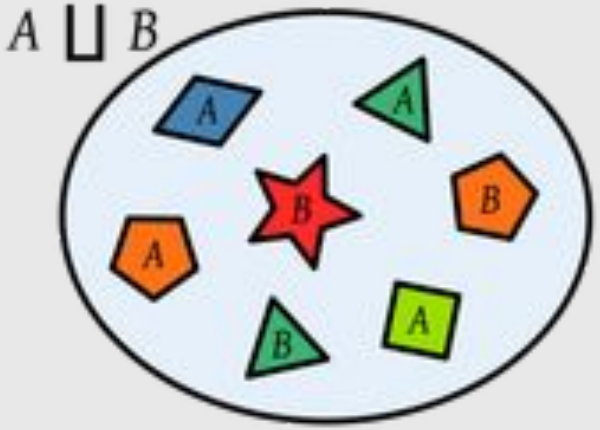
União de conjuntos

Dado vários objetos, gostaríamos de uma estrutura que suporte operações de:

- **Find(u)**: retorna o conjunto do qual ***u*** faz parte.
- **Join(u, v)**: une os conjuntos de ***u*** e ***v*** em um mesmo conjunto.

Com base em Find, podemos ainda ver se dois elementos estão no mesmo conjunto.

- **Same(u, v)**: return Find(u) == Find(v)



União de conjuntos

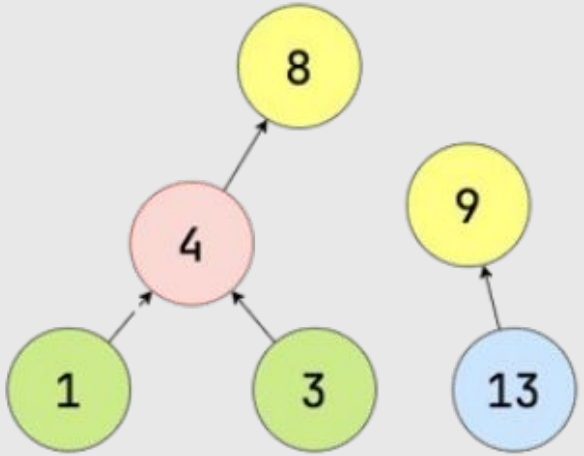
Solução "ingênua":

Podemos pensar em cada objeto como um **vértice** e cada conjunto como uma **árvore** da floresta formada.

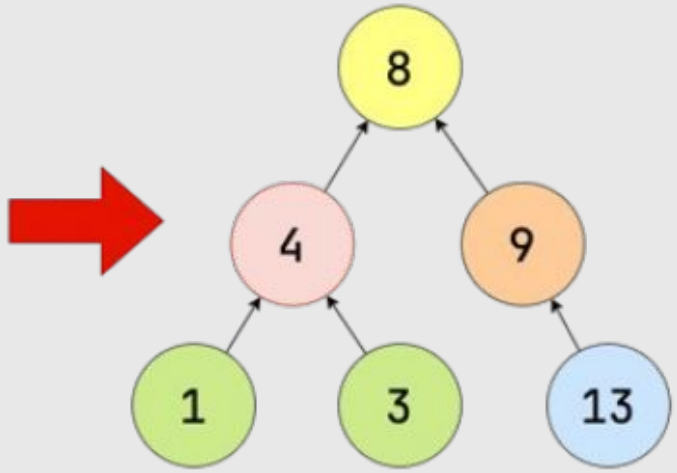
Todo nó aponta para seu **pai** na árvore (exceto a raiz) e a raiz da árvore é o **representante** do conjunto.

No ***Find*** subimos do nó atual até a raiz.

No ***Join*** escolhemos a raiz de uma das árvores e tornamos filha da raiz da outra árvore.



Join(4, 13)



Código DSU *O(N)*

Vamos começar com uma **implementação inicial** simples, exatamente como descrito antes, sendo o “pai” da raiz ela própria.

Complexidade:

No pior caso, a função find vai passar por todos os nós até chegar à raiz da árvore. Ou seja, ***O(N)***, uma **complexidade ruim**, mas que pode ser melhorada.

```
int pai[MAXN];

int find(int u){
  if(pai[u] == u) return u;
  return find(pai[u])
}

void join(int u, int v){
  u = find(u);
  v = find(v);
  pai[v] = u;
}

int main(){
  for(int u=0; u<MAXN; u++)
    pai[u] = u;
}
```

Código DSU O(LogN)

MaratonaCIn · Centro de Informática UFPE · Universidade Federal de Pernambuco

Otimização 1

Compressão de caminhos: sempre que chamarmos o find, salvamos o novo representante do grupo.

Isso evita percorrer novamente o caminho até a raiz a cada chamada da função.

```
int pai[MAXN];

int find(int u){
  if(pai[u] == u) return u;
  return pai[u] = find(pai[u])
}

void join(int u, int v){
  u = find(u);
  v = find(v);
  pai[v] = u;
}

int main(){
  for(int u=0; u<MAXN; u++)
    pai[u] = u;
}
```

Código DSU O(LogN)

Otimização 2

União por tamanho: ao unir duas árvores, ligamos a menor delas à maior. (Small to Large).

Para os nós da árvore grande o caminho não muda. Para a pequena, aumenta em um, mas o novo componente terá pelo menos o dobro do tamanho do antigo, logo só pode aumentar log N vezes.

```
int pai[MAXN], sz[MAXN];

int find(int u){
  if(pai[u] == u) return u;
  return find(pai[u])
}

void join(int u, int v){
  u = find(u);
  v = find(v);

  if(u == v) return;
  if(sz[u] < sz[v]) swap(u, v);

  pai[v] = u;
  sz[u] += sz[v];
}

int main(){
  for(int u=0; u<MAXN; u++)
    pai[u] = u, sz[u] = 1;
}
```

Código DSU O(5)*

Ao unir ambas as otimizações alcançamos a complexidade de **O(α(n))**, ou "função inversa de Ackerman". **α(1e18) = 5**, sendo *praticamente* "constante" para aplicações em programação competitiva.

Mas, **normalmente, só uma das otimizações já é suficiente** para a maioria dos problemas.

MaratonaCIn

Centro de Informática UFPE

Universidade Federal de Pernambuco

```
int pai[MAXN], sz[MAXN];

int find(int u){
  if(u == pai[u]) return u;
  return pai[u] = find(pai[u]);
}

void join(int u, int v){
  u = find(u);
  v = find(v);

  if(u == v) return;
  if(sz[u] < sz[v]) swap(u, v);

  pai[v] = u;
  sz[u] += sz[v];
}

int main(){
  for(int u=0; u<MAXN; u++)
     pai[u] = u,
     sz[u]  = 1;
}
```

MaratonaCIn

Centro de Informática UFPE

Universidade Federal de Pernambuco

Árvore Geradora Mínima

--

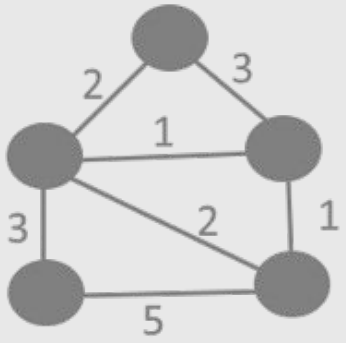
MST

MST

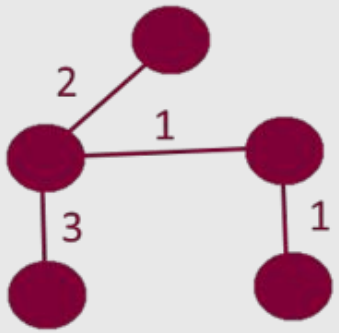
Dado um grafo não direcionado, uma *árvore geradora* é qualquer **subconjunto de arestas do grafo original que forma uma árvore**.

Dado um grafo ponderado, a árvore geradora mínima é uma árvore geradora tal que a **soma dos custos é a menor possível**.

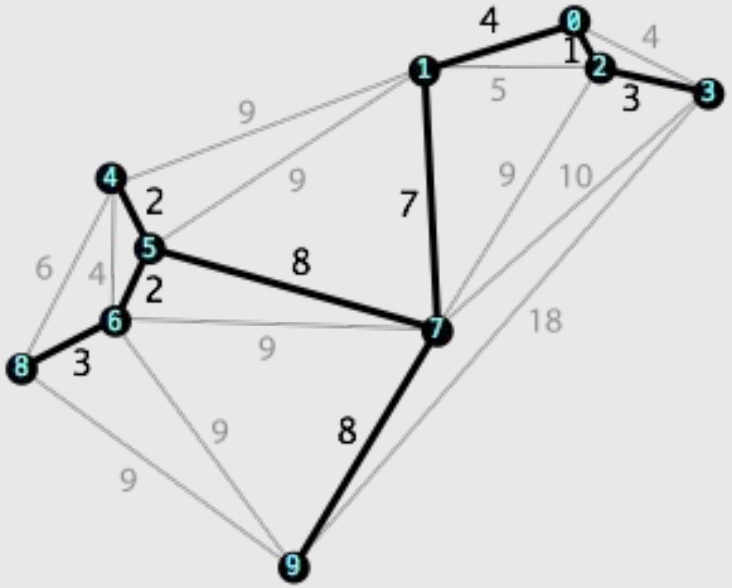
- **Outras propriedades**
  - Se os pesos forem únicos, a mst é única
  - A mst minimiza a custo da maior aresta entre quaisquer nós



Graph



Minimum Spanning Tree, Cost = 7



Kruskal

Para encontrar uma MST podemos usar o algoritmo guloso de kruskal.

Iterando pelas **arestas ordenadas** pelo custo, **se** a aresta atual **conecta componentes distintos, adicionamos ela à resposta**. Isto é, não forma um *ciclo* na árvore!

Para verificar se um ciclo será formado, usamos **DSU**!

```
vector<tuple<int, int, int>> arestas;
// [...]
sort(arestas.begin(), arestas.end());

ll totalCost = 0;
for(auto [cost, u, v] : arestas){
  if( find(u) != find(v) ){
    join(u, v);
    totalCost += cost;
  }
}
```

Kruskal - Prova

Seja S_i o conjunto de arestas escolhidas pelo algoritmo até i-ésima iteração e assuma que existe uma MST T que contém S_i (*S_i é válido*). Na (i+1)-ésima iteração:

1. Se **não escolher**, a aresta forma ciclo e não está em T: ***S_{i+1} é válido.***
2. Se **adiciona a aresta e ela <u>está</u> em T**: Então ***S_{i+1} é válido.***
3. Se **adiciona a aresta e ela <u>não</u> está em T**: Um ciclo seria formado por E.

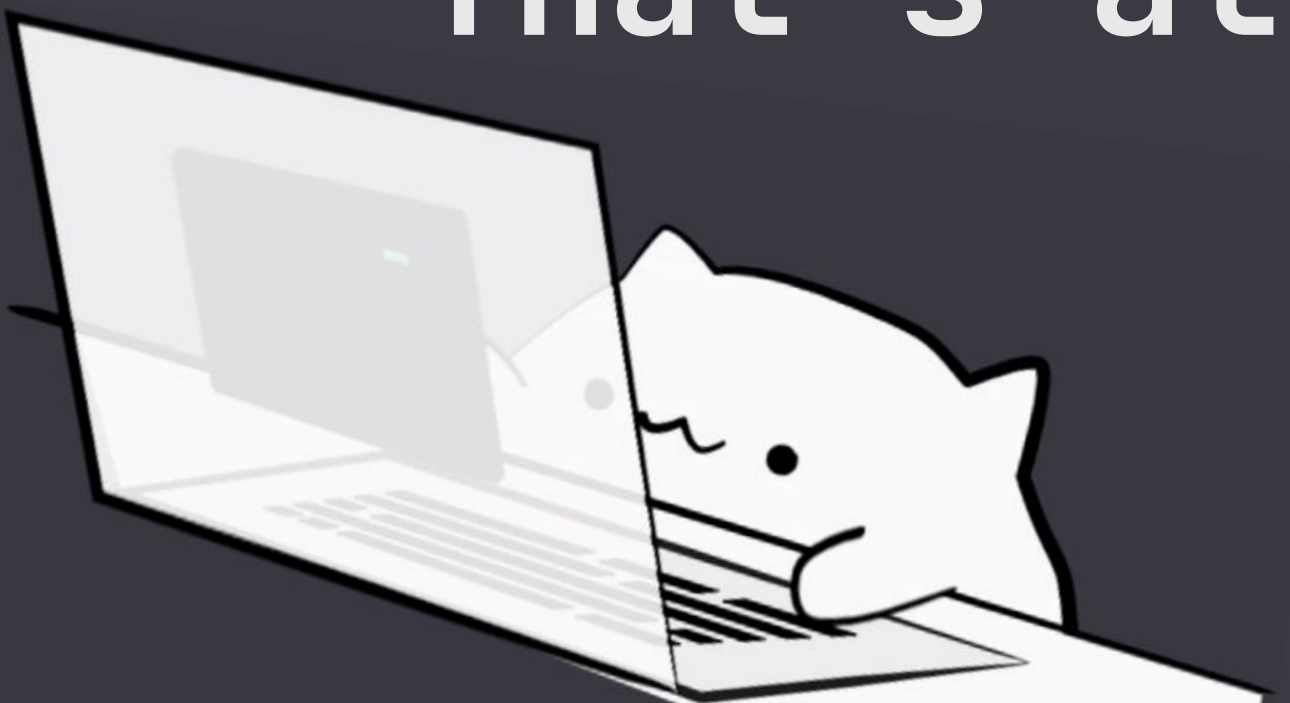
Seja F uma aresta em T no caminho entre os nós U e V que E conecta:

- Se o custo de **F** for **menor que** o custo de **E**: o algoritmo teria visto F antes e, se não escolheu, então F formaria um ciclo. ***Contradição***.
- Se o custo de **F** for **maior que** o custo de **E**, então ao remover F e adicionar E teríamos uma spanning tree com custo menor. ***Contradição***.
- Se o custo de **F** for **igual** ao custo de **E**, então ao remover F e adicionar E temos uma spanning tree com o mesmo custo de T, logo ***S_{i+1} é válido***.

Dúvidas?

MaratonaCIn

Centro de Informática UFPE

Universidade Federal de Pernambuco

That’s all, Folks!